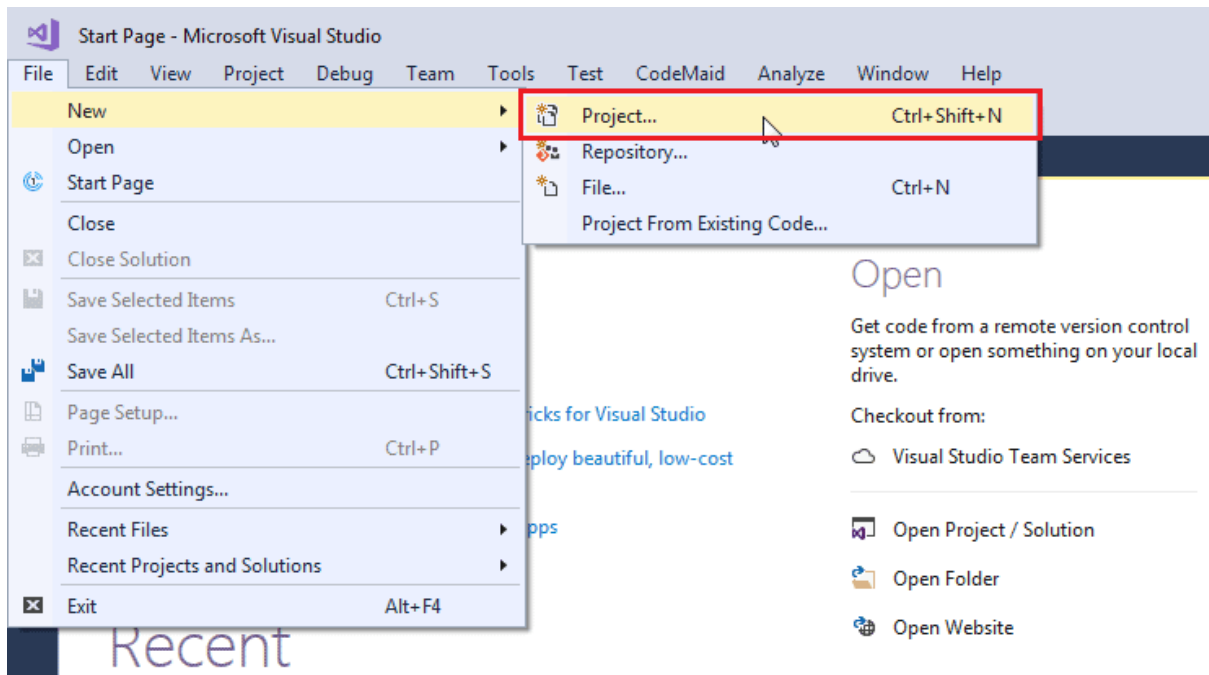


Lesson 2: First C# Program

Here, you will learn to create a simple console application in C# and understand the basic building blocks of a console application.

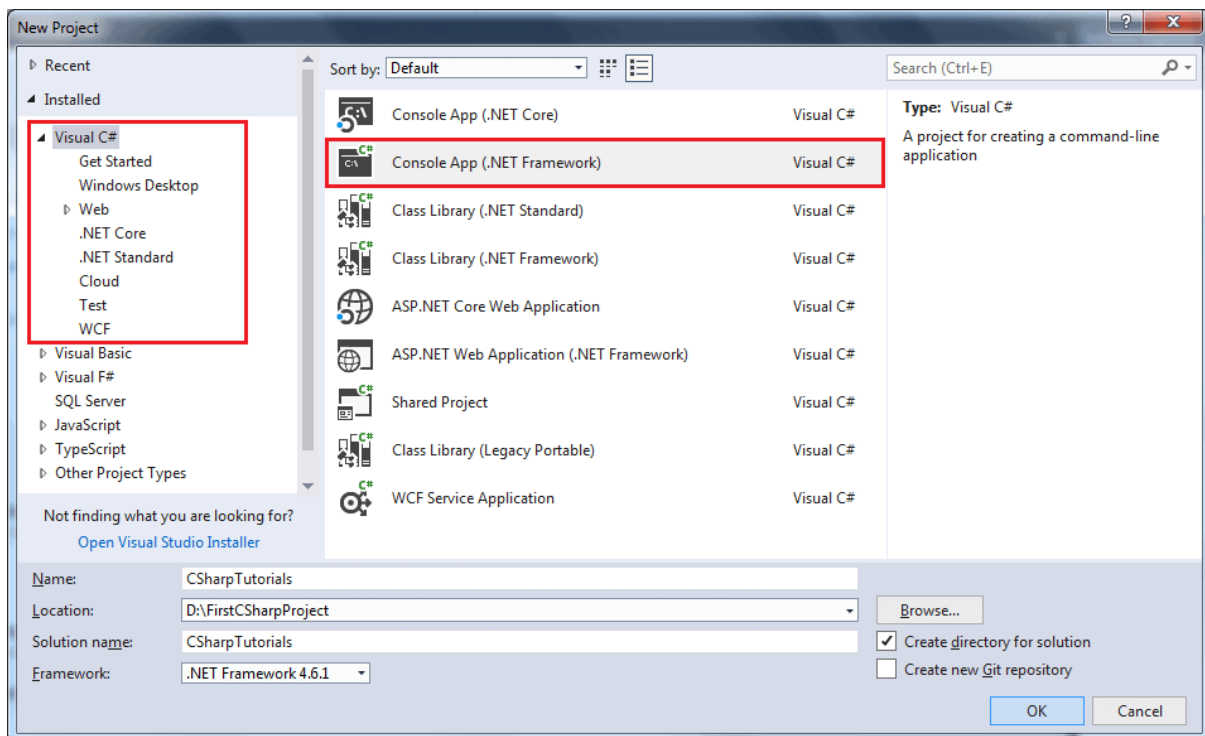
C# can be used in a window-based, web-based, or console application. To start with, we will create a console application to work with C#.

Open Visual Studio (2017 or later) installed on your local machine. Click on File -> New Project... from the top menu, as shown below.



Create a New Project in Visual Studio 2019

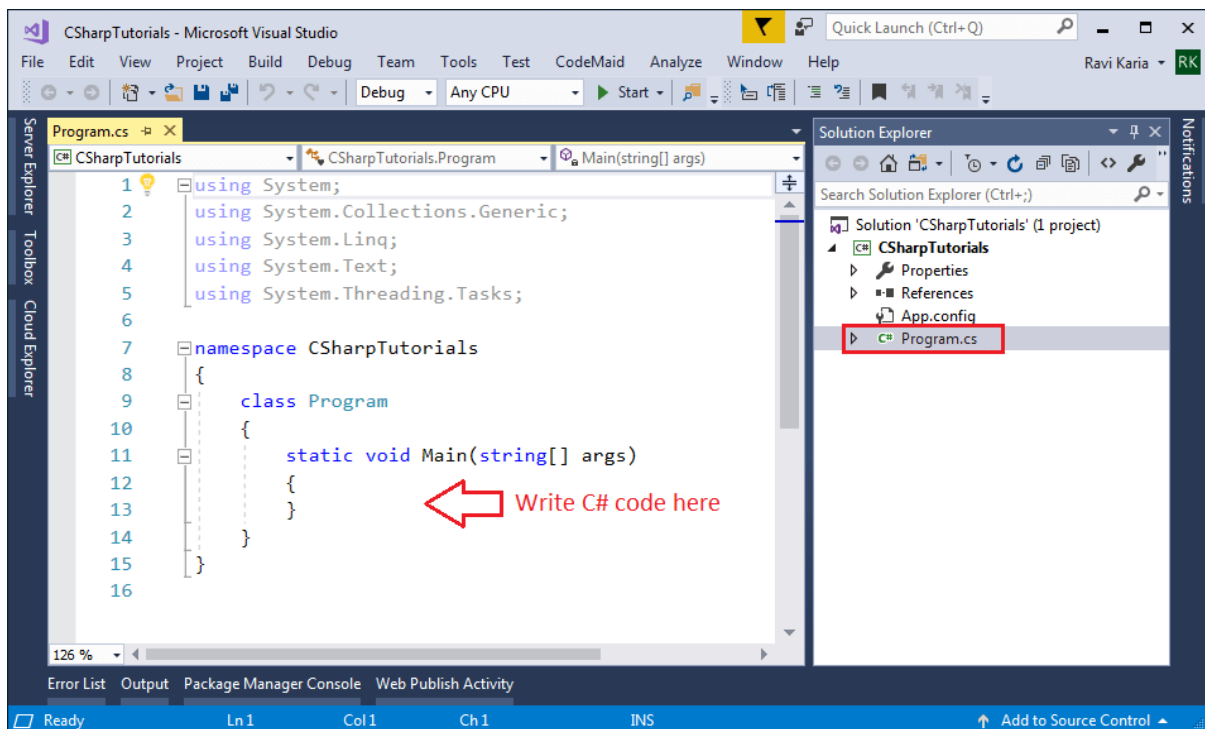
From the **New Project** popup, shown below, select Visual C# in the left side panel and select the Console App in the right-side panel.



Select Visual C# Console App Template

In the name section, give any appropriate project name, a location where you want to create all the project files, and the name of the project solution.

Click OK to create the console project. **Program.cs** will be created as default a C# file in Visual Studio where you can write your C# code in Program class, as shown below. (The .cs is a file extension for C# file.)



C# Console Program

Every console application starts from the Main() method of the Program class. The following example displays "Hello World!!" on the console.

Example: C# Console Application

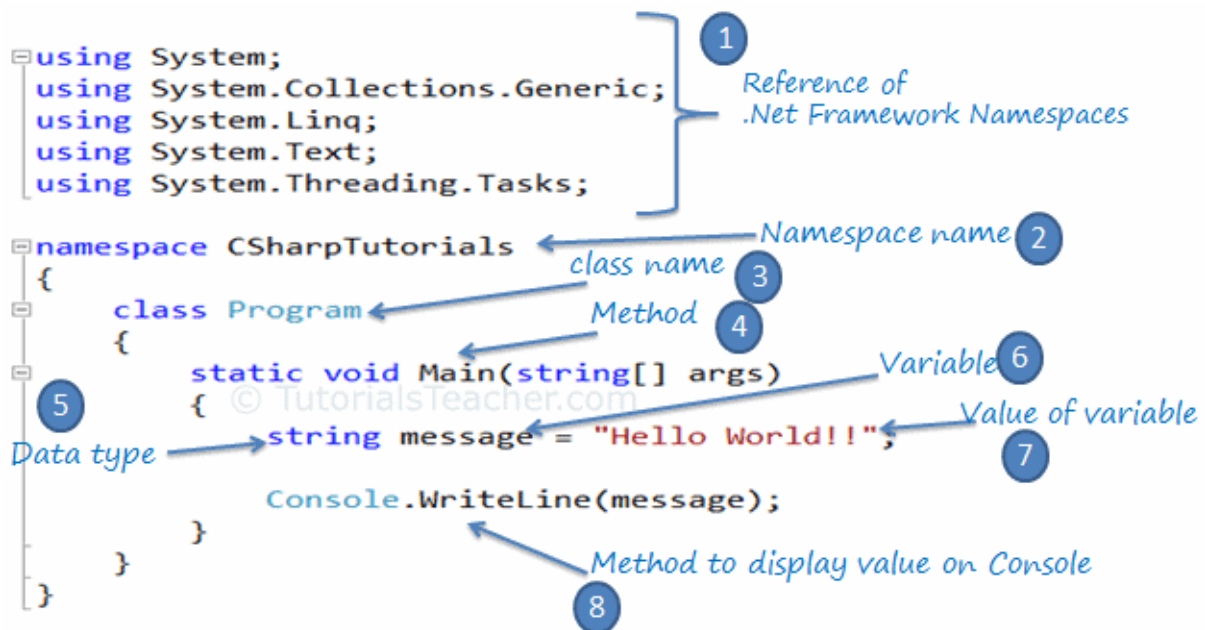
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CSharpTutorials
{
    class Program
    {
        static void Main(string[] args)
        {
            string message = "Hello World!!";

            Console.WriteLine(message);
        }
    }
}
```

[Try it](#)

The following image illustrates the important parts of the above example.



C# Code Structure

Let's understand the above C# structure.

1. Every .NET application takes the reference of the necessary .NET framework namespaces that it is planning to use with the `using` keyword, e.g., `using System.Text`.
2. Declare the namespace for the current class using the `namespace` keyword, e.g., `namespace CSharpTutorials.FirstProgram`
3. We then declared a class using the `class` keyword: `class Program`
4. The `Main()` is a method of `Program` class is the entry point of the console application.
5. `String` is a data type.
6. A `message` is a [variable](#) that holds the value of a specified [data type](#).
7. `"Hello World!!"` is the value of the `message` variable.
8. The `Console.WriteLine()` is a static method, which is used to display a text on the console.

Note:

Every line or statement in C# must end with a semicolon (;).

Compile and Run C# Program

To see the output of the above C# program, we have to compile it and run it by pressing `Ctrl + F5` or clicking the `Run` button or by clicking the `"Debug"` menu and clicking `"Start Without Debugging"`. You will see the following output in the console:

Output:

Hello World!!

Casting and Type Conversion in C#

difference between casting and type conversion in C#?

- Casting is the process of converting one data type into another. In C#, **Type casting can be done in two ways: implicit and explicit casting**. **Type conversion in C# can be performed using built-in methods**. This is generally needed when converting between incompatible types, such as from a string to an integer.

Casting in C#

- Casting is the process of **converting one data type into another**. In C#, casting can be done in **two ways: implicit and explicit casting**.

1. Implicit Casting

- Implicit casting (also known as **automatic conversion**) happens when the compiler can perform the conversion automatically without the risk of data loss. It is typically possible between compatible types and where the target type has a larger capacity.

```
C#  
  
int myInt = 10;  
double myDouble = myInt; // An integer can be implicitly cast to a double  
  
Console.WriteLine(myDouble);  
// Output: 10
```

2. Explicit Casting

- Explicit casting, sometimes called type casting or forced conversion, is necessary when there's potential for data loss - typically when you're attempting to convert a larger type to a smaller one.

```
double myDouble = 9.78;
int myInt = (int) myDouble; // A double is explicitly cast to an integer

Console.WriteLine(myInt);
// Output: 9
```

Type Conversion in C#

Type conversion in C# can be performed using built-in methods. This is generally needed when converting between incompatible types, such as from a string to an integer.

Here's an example of type conversion using the Convert class:

```
string myString = "10";
int myInt = Convert.ToInt32(myString); // Convert string to integer

Console.WriteLine(myInt);
// Output: 10
```

Math and user input in c#

Simple exercise 1

How to find The area of a circle.

using System;

namespace MathExampleOne

```
{
    class Program
    {
        static void Main(string[] args)
        {
            double pi = 3.14159;
            double radius = 2.5;
            double area = pi * radius * radius;
            Console.WriteLine("The area of a circle with radius " +
radius + " is " + area);
            Console.WriteLine("Hit any key to end...");
            Console.ReadKey();
        }
    }
}
```

```
    }  
  }  
}
```

output

The area of a circle with radius 2.5 is 19.6349375
Hit any key to end...

User Input Example

A programmer can request the user to input a value in the console. The easiest method to read from the console is to read the console input until the user hits the **enter** button. A line must be stored in a C#

using System;

namespace VariablesExampleTwo

```
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            double pi = 3.14159;  
            Console.Write("Please enter a value for the radius: ");  
            double radius = double.Parse(Console.ReadLine());  
            double area = pi * radius * radius;  
            Console.WriteLine("The area of a circle with radius " +  
radius + " is " + area);  
            Console.WriteLine("Hit any key to end...");  
            Console.ReadKey();  
        }  
    }  
}
```

Output

Please enter a value for the radius: 2.5
The area of a circle with radius 2.5 is 19.6349375
Hit any key to end...

Remarks

```
Console.WriteLine("Please enter a value for the radius: ");  
double radius = double.Parse(Console.ReadLine());
```

code Analysis

- First we prompt the user for a value. This ensures that the user realizes that the program is waiting for input. Then we read a **double** into the variable named **radius**. This radius is used to compute the area of the circle. Note that this program will not deal with any errors that could be generated by the Parse() method.

Operators are symbols that can be used with operands to perform tasks.

- Operators are symbols that can be used with operands to perform tasks.

category	Operators	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Right to left
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Right to left
Conditional	?:	Right to left
Assignment	- += -= *-= /-= %-= >>= <<= &= ^= =	Right to left


```

using System;
namespace OperatorsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Result;
            int Num1 = 20, Num2 = 10;

            // Addition Operation
            Result = (Num1 + Num2);
            Console.WriteLine($"Addition Operator: {Result}" );

            // Subtraction Operation
            Result = (Num1 - Num2);
            Console.WriteLine($"Subtraction Operator: {Result}");

            // Multiplication Operation
            Result = (Num1 * Num2);
            Console.WriteLine($"Multiplication Operator: {Result}");

            // Division Operation
            Result = (Num1 / Num2);
            Console.WriteLine($"Division Operator: {Result}");

            // Modulo Operation
            Result = (Num1 % Num2);
            Console.WriteLine($"Modulo Operator: {Result}");

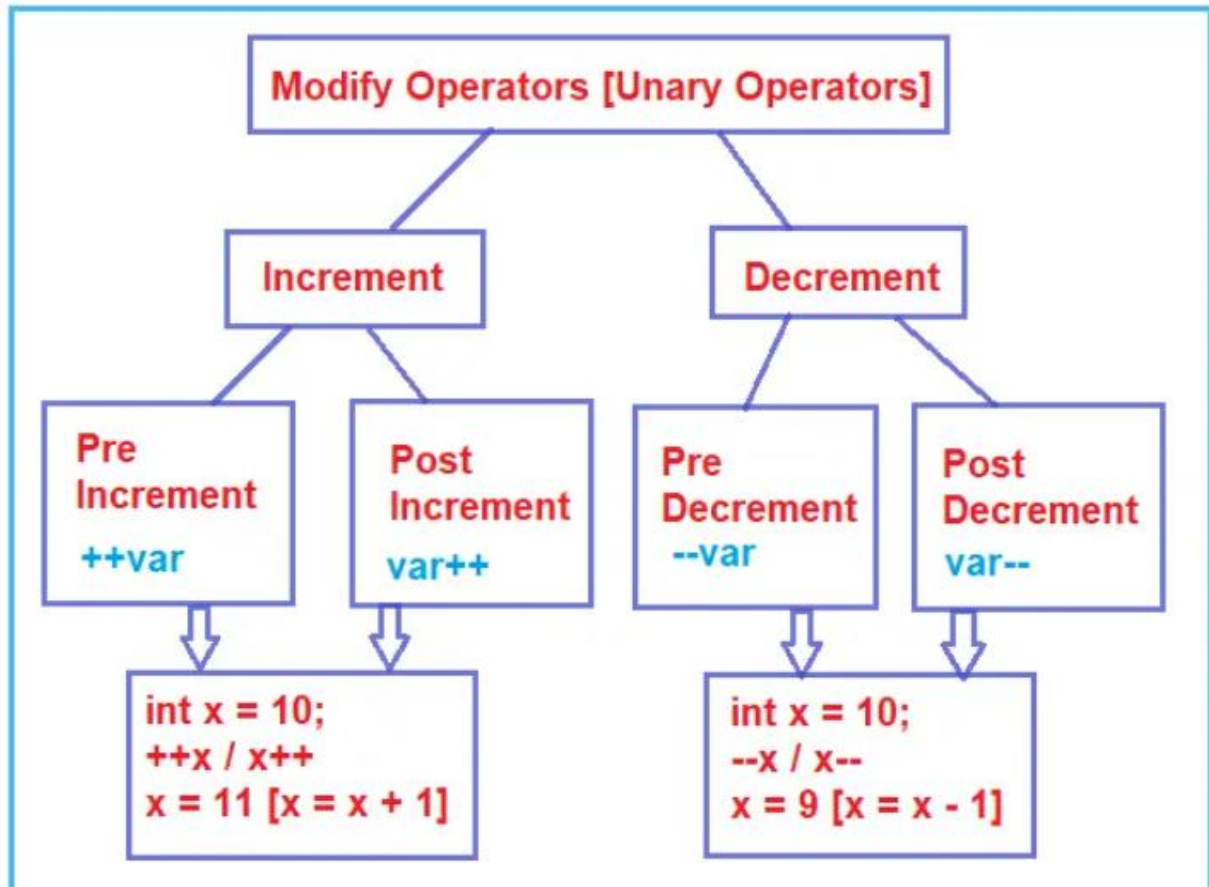
            Console.ReadKey();
        }
    }
}

```

```

Add Assignment Operator: 25
Subtract Assignment Operator: 15
Multiply Assignment Operator: 75
Division Assignment Operator: 5
Modulo Assignment Operator: 0

```



```
using System;
namespace OperatorsDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Post-Increment
            int x = 10;
            // Result1 is assigned 10 only,
            // x is not updated yet
            int Result1 = x++;
            //x becomes 11 now
            Console.WriteLine("x is {0} and Result1 is {1}", x, Result1);

            // Pre-Increment
            int y = 10;
            int Result2 = ++y;
            //y and Result2 have same values = 11
            Console.WriteLine("y is {0} and Result2 is {1}", y, Result2);

            Console.ReadKey();
        }
    }
}
```

Output:

```
x is 11 and Result1 is 10  
y is 11 and Result2 is 11
```

Logical Operator

Practice Exercises

1. Use user input and the Math namespace to generate the sin and cos of a user assigned value.
2. Use user input and the Math namespace to find the maximum value of two user assigned values.
3. Write a program to calculate area of triangle. Take required input from user and then calculate the area of triangle.
4. What will be the output of the following C# relational operator code?

```
int n = 2;  
  
    int p = 4;  
    int q = 5;  
    int w = 3;  
  
    if ( !((p * q) / n <= (q * w) + n/p ))  
    {  
        Console.WriteLine( ++p + w++ + " " + ++n);  
        Console.WriteLine("b");  
    }  
  
    else  
    {  
        Console.WriteLine(--p + q-- + " " + --n);  
        Console.WriteLine("a");  
    }
```

}